

Supplementary Appendix for Orion: Spatiotemporal Neural Networks with Self-Validating Directed Graphs for Traffic and Epidemic Forecasting

Anonymous Author(s)

ACM Reference Format:

Anonymous Author(s). 2026. Supplementary Appendix for Orion: Spatiotemporal Neural Networks with Self-Validating Directed Graphs for Traffic and Epidemic Forecasting. In . ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Note to readers. This supplementary appendix accompanies our CIKM 2026 submission and contains extended technical material referenced from the main paper. It is one component of our overall supplementary material (which also includes source code and configuration files). Section numbers are prefixed with “S.” to distinguish from the main paper. References of the form “Equation k of the main paper” or “Table k of the main paper” point to the corresponding objects in the main submission PDF.

S.1 Theoretical Foundations and Proofs

S.1.1 Granger Causality Foundation of TE-CausGAT

We provide the theoretical foundation for TE-CausGAT based on the Granger causality framework [3], which is the appropriate formalism for learning directed predictive dependencies from observational time series data.

Definition (Granger Causality). A time series X is said to Granger-cause another time series Y if, given all other relevant information, past values of X provide statistically significant information about future values of Y . Formally:

$$\sigma^2(Y_t|Y_{t-1}, \dots, X_{t-1}, \dots) < \sigma^2(Y_t|Y_{t-1}, \dots) \quad (1)$$

where $\sigma^2(\cdot|\cdot)$ denotes the conditional variance.

Connection to TE-CausGAT. Our learned directed dependency matrix $C^{(s)} \in [0, 1]^{N \times N}$ captures Granger-causal relationships in a neural network framework. Specifically, $C_{ij}^{(s)} > 0$ indicates that node i ’s historical information improves the prediction of node j in temporal segment s . This interpretation aligns with the extensive literature on neural Granger causality [4, 8].

Prediction-Based Validation Principle. Our validation mechanism tests whether the learned dependency strengths are consistent with their predictive effects. The key insight is based on the *predictive relevance criterion*: if removing node i ’s contribution significantly changes the prediction of node j , then node i carries

predictively relevant information for node j . This is operationalized through our validity score (Equation 5 of the main paper), where $\Delta\hat{y}_j$ measures the prediction change when node i ’s contribution is masked.

Remark on Scope. We emphasize that Granger causality captures *predictive* rather than *interventional* causality. Our method identifies which nodes carry predictive information for other nodes, which is sufficient for forecasting applications. We do not claim to recover the underlying structural causal model or to predict the effects of real-world interventions.

S.1.2 Consistency Between Learned Dependencies and Predictive Effects

We analyze the conditions under which TE-CausGAT learns dependency structures that are consistent with their predictive effects.

Proposition (Prediction Consistency). Under the assumption that the model has sufficient capacity to capture the true predictive relationships, the learned dependency matrix C converges to values where high-weight edges correspond to high predictive importance.

Justification. Consider the optimization objective in Stage 2 and Stage 3 training:

$$\mathcal{L} = \mathcal{L}_{pred} + \lambda_{validity} \mathcal{L}_{validity} + \dots \quad (2)$$

The validity loss $\mathcal{L}_{validity}$ penalizes inconsistency between C_{ij} and the normalized prediction change $\Delta\hat{y}_j / \max_k(\Delta\hat{y}_k)$. Minimizing this loss drives the model toward configurations where:

- Edges with high learned weights C_{ij} correspond to node pairs where masking node i significantly affects the prediction of node j ;
- Edges with low learned weights correspond to node pairs with minimal predictive coupling.

This self-consistency mechanism helps filter out spurious correlations that may arise from confounding factors or noise. While we cannot guarantee identification of true causal relationships in the interventional sense, this mechanism ensures that the learned structure reflects genuine predictive dependencies that are useful for forecasting.

Empirical Validation. The effectiveness of this consistency mechanism is validated through:

- **Shuffling Test (Section S.1.4):** When node-effect correspondences are randomly shuffled, both validity scores and prediction performance degrade significantly, confirming that the model learns genuine node-specific dependencies rather than trivial patterns.
- **DREAM-3 Benchmark (Table 6 of the main paper):** On a dataset with known ground-truth regulatory relationships, Orion achieves the highest AUC among all baselines,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

demonstrating that learned dependencies align with true directed relationships.

S.1.3 Practical Robustness of the Validation Mechanism

Our prediction-based validation mechanism is designed with multiple robustness enhancements to ensure reliable learning of directed dependencies on complex real-world data.

While our theoretical derivation assumes local linearization, our implementation fully captures nonlinear dynamics through multi-scale temporal convolutions (8 different kernel sizes) and GRU propagation mechanisms. The multi-scale temporal convolution module in our code employs multiple time scales from 1 to 15, ensuring simultaneous modeling of short-term perturbations and long-term trends. The normalization term $\max_k(\Delta\hat{y}_k)$ combined with the stability term $\epsilon_{\text{stab}} = 10^{-8}$ not only prevents numerical instability but more importantly provides relative scale calibration of node responses, making causal strengths comparable across nodes of different magnitudes.

Our implementation through prior-guided fusion (Equation 4 of the main paper) intelligently combines data-driven directed dependency learning with domain prior knowledge. When data evidence is sufficient, the model relies on learned directed dependencies; when uncertainty exists, prior knowledge provides guidance. This adaptive mechanism ensures our method remains robust under various data conditions.

The InterventionValidator class in our code implements an intelligent intervention node rotation mechanism, ensuring all nodes are systematically tested. By constraining intervention values within $[-1, 1]$ through $v = \tanh(g(c))$, we maintain numerical stability while adequately exploring the intervention space. The three-stage progressive training strategy (feature learning $\rightarrow$ directed structure learning $\rightarrow$ end-to-end optimization) further ensures reliable learning of directed structures.

Our method achieves 6.60% MAE improvement on METR-LA, 8.65% improvement on PEMS-08, and remarkable 22.70% and 37.73% improvements on CA-COVID and TX-COVID respectively. These results demonstrate the advantage of our Granger-inspired directed dependency learning approach over pure correlation-based methods, by identifying predictively relevant directed dependencies rather than superficial correlations, we achieve more accurate and interpretable predictions.

S.1.4 Safeguards Against Confirmation Bias

To prevent confirmation bias, our validation mechanism includes three layers of independence guarantees: (1) intervention nodes are randomly selected in each batch, independent of the current dependency matrix; (2) intervention effects propagate through an independently trained GRU, whose dynamics do not directly depend on the learning of C; (3) the validity score uses normalized relative effects, requiring the model to predict the correct ranking of causal strengths rather than absolute values. Stress test experiments show that after shuffling node-effect correspondences, the model's validity in late training significantly decreases (average 1.4%, $p < 0.05$) and prediction MAE degrades by 3.84%, validating

that the mechanism achieves high validity scores through learning predictively relevant directed dependencies rather than trivial alignment.

To verify that the perturbation mechanism does not exhibit confirmation bias, we designed a Shuffling Test to examine whether the model truly learns the dependency correspondences between nodes. The test protocol is as follows: at each test epoch, we perform the standard intervention process on the current model to obtain the intervention effects $\Delta\hat{y}_j$ for each node, then randomly shuffle these effect values across different nodes and recalculate the validity score. If the model relies on genuine directed dependencies, validity should significantly decrease after shuffling; if trivial alignment exists, there should be no significant difference before and after shuffling. We conduct tests every 5 training epochs on the PEMS-08 dataset, performing 30 random shuffles each time to calculate statistical significance (using paired t-test with significance level $p < 0.05$).

From Table 1, a clear two-stage pattern is evident: during early training (Epoch 10), validity slightly increases after shuffling (negative drop magnitude) and is statistically insignificant, indicating that causal relationships have not been sufficiently learned at this stage. Starting from Epoch 15, validity consistently decreases after shuffling with statistical significance, proving that the model has learned genuine node-effect dependency pairings. Notably, the Week and Day cycles maintain significant validity drops (0.98%-2.47%) throughout the training process, while the Hour cycle's drop approaches zero by Epoch 30, reflecting the convergence characteristics of directed dependency learning at different time scales.

Table 1: Shuffling Test Results Throughout Training Process (PEMS-08). Drop magnitude = (Original - Shuffled)/Original.

Epoch	Period	Original	Shuffled	Drop
15	Hour	0.1432	0.1420	0.84%
	Week	0.1467	0.1434	2.28%
	Day	0.1493	0.1465	1.85%
25	Hour	0.1414	0.1409	0.33%
	Week	0.1470	0.1453	1.16%
	Day	0.1504	0.1480	1.58%

From Table 2, after shuffling causal correspondences, the model trained on the test set shows an increase in MAE from 13.29 to 13.80 (3.84%) and RMSE from 20.24 to 21.33 (5.39%). This performance degradation directly shows that the original model's excellent prediction performance depends on correctly learned directed dependencies, rather than trivial alignment. Notably, MAPE slightly decreases (-4.36%), possibly because the shuffled model has reduced prediction bias at extremely low traffic values (where denominators close to zero cause MAPE instability), but significant degradation in overall MAE and RMSE still indicates the model's dependence on correctly learned directed structures.

Combining evidence from both validity decrease and prediction performance degradation, our stress test validates the effectiveness of the perturbation mechanism: the model must learn genuine inter-node directed dependencies to simultaneously achieve high validity

Table 2: Impact of Shuffling Test on Prediction Performance

Model Configuration	MAE	RMSE	MAPE(%)	Performance Change
Original Model (No Shuffling)	13.29	20.24	10.55	Baseline
Shuffled Model	13.80	21.33	10.09	MAE↑3.84%, RMSE↑5.39%

scores and excellent prediction performance, rather than through self-confirming trivial alignment.

S.2 DAG Constraint Effectiveness Analysis

We adopt the DAG constraint in polynomial approximation form:

$$\mathcal{L}_{\text{DAG}} = \lambda_2 \cdot \text{tr}(\hat{C}^2) + \lambda_3 \cdot \text{tr}(\hat{C}^3) \quad (3)$$

where $\hat{C} = C_{\text{fused}} / \|C_{\text{fused}}\|_{\infty}$ is normalized, with $\lambda_2 = 0.1$, $\lambda_3 = 0.01$. The main reasons for choosing this approximation are computational efficiency and numerical stability: the complete matrix exponential constraint $\text{tr}(e^C) - N = 0$ has computational complexity of $O(N^3)$ for large-scale networks (METR-LA has 207 nodes, PEMS-04 has 307 nodes) and is prone to gradient explosion or vanishing during backpropagation. In contrast, the polynomial approximation only requires computing the trace of C^2 and C^3 , with reduced complexity and smaller constants, making it numerically more stable. Theoretically, $\text{tr}(C^k)$ computes the number of cycles of length k in the graph. By penalizing $\text{tr}(C^2)$ and $\text{tr}(C^3)$, we suppress the most common 2-cycles (bidirectional edges) and 3-cycles, which in practice can effectively constrain the acyclicity of the graph. Although this cannot theoretically guarantee complete acyclicity (as cycles of length ≥ 4 may exist), we observe in our experiments that this constraint is sufficiently effective.

To quantitatively verify the actual effectiveness of the DAG constraint, we use topological sorting algorithms to detect cycles in the learned directed dependency graph on the PEMS-08 dataset (170 nodes). Under the condition of setting threshold=0.1 (used to binarize the continuous dependency strength matrix), Table 3 presents the complete cycle statistics:

Table 3: Cycle statistics of learned directed dependency graph (PEMS-08, threshold=0.1).

Cycle Type	Count	Theoretical Max	Pct.
2-cycles (bidirectional)	0	14,365 ($\binom{170}{2}$)	0.00%
3-cycles	0	804,440 ($\binom{170}{3}$)	0.00%
4-cycles and longer	0	—	0.00%

The results show that at threshold 0.1, the learned directed dependency graph completely eliminates all cycles of any length (including 2-cycles, 3-cycles, and 4-cycles and longer), strictly satisfying the DAG property. This verifies the sufficiency of the polynomial approximation $\lambda_2 \cdot \text{tr}(C^2) + \lambda_3 \cdot \text{tr}(C^3)$ in practical applications, although theoretically it cannot completely guarantee the absence of long cycles, through appropriate weight settings ($\lambda_2 = 0.1 > \lambda_3 = 0.01$, prioritizing the penalization of short cycles), it can effectively eliminate all cycles in practice.

S.3 Extended Implementation Details

This section provides extended hyperparameter configurations and implementation analyses that complement the main-paper appendix.

S.3.1 LLM-Enhanced Retrieval Comparison Configuration

To control variables and verify the independent contribution of LLM retrieval, we select 5 representative nodes from PEMS08 and METR-LA datasets to construct subsets. The experimental design employs strict variable control: maintaining all configurations identical except for the *use_llm_embedding* parameter. Historical data windows are set to 4 weeks/6 days/1 hour, providing retrieval candidates for the LLM. Model capacity is reduced ($d_{\text{model}} = 8$) to accommodate small-scale networks and avoid overfitting, while intervention frequency is increased to 2 to enhance directed dependency learning effectiveness in small networks. As shown in Table 4, this control ensures that performance differences can be attributed to the LLM retrieval mechanism.

Table 4: LLM comparison experiment configuration (5-node subset)

Parameter	Configuration Value
num_of_vertices	5
num_weeks/days/hours	4/6/1
d_model	8
d_ff*	16
n_belt_block	1
intervention_frequency	2
num_interventions	5
batch_size	128
learning_rate	0.0005
Unique Variable	
use_llm_embedding	False → True

Ablation Experiment Configuration. Ablation experiments are based on the complete PEMS08 configuration, with different components controlled through the *ablation_mode* parameter. We systematically evaluate 8 key components, with all ablation variants maintaining identical training configurations to ensure fair comparison.

S.3.2 Dream3 Directed Dependency Validation Configuration

To quantitatively evaluate Orion’s directed dependency learning capability, we conduct experiments on the DREAM-3 gene regulatory network dataset. Unlike the main application scenario (traffic forecasting), DREAM-3 is configured as a static causal discovery task: we set the number of temporal segments $S = 1$ to learn a single time-invariant regulatory network $C \in \mathbb{R}^{100 \times 100}$, adopting single-step lag modeling (gene i at time t influences gene j at time $t + 1$), which aligns with the standard setting of the DREAM challenge.

Due to the specificity of the DREAM-3 dataset (only 21 time points), specialized configurations are required to prevent overfitting: (1) a small learning rate (1e-5) and a lightweight model ($d_model=8$) are used to adapt to the small data scale (21 time points $\times$ 100 samples); (2) temporal segments are set to 1 since gene expression data lacks daily periodicity; (3) the dependency threshold is set to 0.05, stricter than traffic data (0.1), to avoid false positive edges in biological networks. These configurations (Table 5) are specifically designed for the unique characteristics of biological time series data.

Table 5: Dream3 directed dependency validation experiment configuration

Parameter	Configuration Value
num_of_vertices	100
learning_rate	1e-5
d_model	8
num_time_segments	1
batch_size	16
dependency_threshold	0.05

The perturbation validation mechanism is still employed in this scenario ($K=25$ genes randomly intervened, with effects propagated through GRU). The training process does not use the ground truth network, and evaluation metrics such as AUC are calculated only during assessment using the official DREAM scripts. All DREAM-3 experiments were conducted on an NVIDIA RTX 2080Ti (22GB) GPU.

S.3.3 Three-Stage Training Weight Configuration

The three-stage training strategy employs carefully calibrated loss component weights to progressively transition from feature learning to directed dependency discovery. Table 6 details the individual loss component weights during Stage 2:

$$\mathcal{L}_{causal_total} = w_{sparse}\mathcal{L}_{sparse} + w_{valid}\mathcal{L}_{validity} + w_{conn}\mathcal{L}_{connect} + w_{temp}\mathcal{L}_{temporal} + w_{dag}\mathcal{L}_{dag} \quad (4)$$

Table 7 presents the complete loss weight evolution across all three stages.

Table 6: Directed dependency loss component weights in Stage 2.

Causal Loss Component	Weight (w_i)
Sparsity ($\mathcal{L}_{sparse}$)	0.01-0.02*
Validity ($\mathcal{L}_{validity}$)	0.5
Connectivity ($\mathcal{L}_{connect}$)	0.1
Temporal ($\mathcal{L}_{temporal}$)	0.01
DAG ($\mathcal{L}_{dag}$)	0.1

*Adaptive: 0.01 when avg validity >0.5, else 0.02

The sparsity weight (*) is dynamically adjusted based on validity scores: when average validity exceeds 0.5, the weight decreases to

Table 7: Loss weight distribution across training stages

Loss Component	Stage 1	Stage 2	Stage 3
Prediction ($\mathcal{L}_{pred}$)	1.0	1.0	1.0
Sparsity	0	0.0005	0.0003
Validity	0	0.025	0.009
Connectivity	0	0.005	0.0015
Temporal	0	0.0005	0.00015
DAG	0	0.005	0.0015
Weight Decay	0.0001	0.0001	0.0001

0.01 to encourage more connections; otherwise, it increases to 0.02 to enforce sparsity. This adaptive mechanism balances between discovering sufficient directed dependencies and avoiding spurious connections.

S.3.4 K-means Node Selection Strategy

Due to the computational complexity of LLM retrieval being $O(N \times K_{hist})$, where N is the number of nodes and K_{hist} is the number of historical candidates, direct application on large-scale networks would incur prohibitive computational overhead. Therefore, we employ K-means clustering to select representative node subsets for LLM comparison experiments. All LLM-enhanced retrieval experiments were implemented on an NVIDIA RTX 5880 Ada (48GB) GPU with Intel Xeon 6330 CPU and 107GB RAM.

The node selection process comprises four steps:

Step 1: Feature Extraction. For each node i , we extract a four-dimensional feature vector from its time series data:

$$\mathbf{f}_i = [\mu_i, s_i, \rho_i(12), r_i^{peak}] \quad (5)$$

where μ_i and s_i are the mean and standard deviation of node i 's traffic flow respectively, $\rho_i(12)$ is the autocorrelation coefficient at lag 12 steps (1 hour) capturing short-term periodicity, and r_i^{peak} is the peak hour flow ratio reflecting the node's functional characteristics.

Step 2: K-means Clustering. We execute standard K-means algorithm on the feature matrix $\mathbf{F} = [\mathbf{f}_1, \dots, \mathbf{f}_N]^T$, setting cluster number $k = 5$ to obtain cluster centers $\mu = \{\mu_1, \dots, \mu_5\}$.

Step 3: Representative Node Selection. From each cluster, we select the node closest to the centroid as representative:

$$n_j^* = \arg \min_{n \in S_j} \|\mathbf{f}_n - \mu_j\|_2 \quad (6)$$

where S_j denotes the node set of the j -th cluster.

Step 4: Connectivity Verification. We examine the connectivity of selected nodes in the adjacency matrix to ensure the subnetwork maintains certain spatial connectivity.

As shown in Table 8, the METR-LA subset contains 3 isolated nodes (nodes 29, 67, 127 with no edge connections), reflecting K-means' characteristic of selecting based on temporal features rather than spatial topology. This selection strategy ensures diversity in temporal patterns, providing comprehensive test scenarios for LLM-enhanced retrieval—encompassing both spatially correlated node pairs and spatially independent nodes with similar temporal patterns, thereby thoroughly evaluating LLM retrieval effectiveness across different scenarios.

Table 8: K-means node selection results for PEMS08 and METR-LA datasets

New ID	PEMS08		METR-LA	
	Original ID	Connectivity	Original ID	Connectivity
0	9	Connected	29	Isolated
1	10	Connected	67	Isolated
2	25	Connected	71	Connected
3	31	Connected	127	Isolated
4	166	Connected	156	Connected

S.3.5 Multi-Scale Temporal Convolution Ablation

We conducted systematic ablation experiments on the selection of kernel numbers in multi-scale temporal convolution to explain the question of “why we choose 8 kernels”.

Experimental Setup: All experiments were conducted on the PEMS-08 dataset, keeping all hyperparameters consistent except for the kernel configuration ($d_model=64$, $n_belt_block=2$, $batch_size=128$, $learning_rate=0.005$). We tested four kernel configurations: (1) Single-3: a single kernel size $k=3$, serving as a baseline without multi-scale modeling; (2) Sparse-4: sparse sampling of 4 key scales [1,3,7,15], testing whether fewer kernels can cover key patterns; (3) Full-8: dense sampling of 8 odd scales [1,3,5,7,9,11,13,15]; (4) Dense-12: ultra-dense sampling of 12 consecutive scales [1,2,3,...,12], testing whether more kernels bring further improvements. All models were trained for 100 epochs using a three-stage training strategy on NVIDIA A100-40GB GPUs.

As shown in Table 10, the experimental results clearly demonstrate the multi-dimensional impact of kernel configuration on model performance. First, Single-3 shows significant performance degradation compared to Full-8 across all prediction horizons, particularly at long horizon (Horizon 12) where MAE increases from 15.18 to 17.99 (degradation of 18.5%) and RMSE from 23.37 to 27.81 (degradation of 19.0%), fully demonstrating that a single temporal scale cannot adequately capture the complex periodic and trend patterns in spatiotemporal data. Second, although Sparse-4 covers both the minimum ($k=1$) and maximum ($k=15$) scales, its performance still lags behind Full-8 by approximately 6.92% in MAE, indicating that intermediate scales ($k=5,9,11,13$) are equally critical for capturing temporal patterns between immediate response and long-term trends (such as half-hour fluctuations) and cannot be simply omitted.

In terms of computational efficiency, the training time of Full-8 (52.7s/epoch) is comparable to Single-3, because the efficient implementation of dilated convolutions allows the computation of multiple kernels to be parallelized; however, the training time of Dense-12 surges to approximately 45 minutes/epoch (2700s), mainly due to the larger parameter count from 12 kernels, the lack of efficiency advantages from dilated structures in consecutive small kernels ($k=2,4,6...$), and increased memory and computational overhead from denser feature fusion, making complete training (100 epochs) require approximately 75 hours, which is unacceptable in practical applications. Finally, we choose odd kernel sizes

(1,3,5,...,15) instead of even numbers to maintain temporal symmetry, because odd kernels have a clear “current” time point at the center moment, while even kernels introduce a half-step temporal offset, which may cause time alignment issues in causal inference.

As shown in Table 9, Full-8 is clearly superior in Pareto efficiency: compared to Single-3, it achieves 7.26% MAE improvement without increasing training time; compared to Sparse-4, it achieves 6.48% further improvement with even shorter training time (benefiting from better parallelization), while the training cost of Dense-12 makes it impractical in real applications.

Table 9: Pareto Analysis of Training Time and Performance

Configuration	MAE	Training Time (h/100epochs)	Efficiency Ratio
Single-3	14.33	1.46	Baseline
Sparse-4	14.21	1.61	-0.08% / +10.3%
Full-8	13.29	1.46	-7.26% / +0%
Dense-12	—	~75.00	— / +5000%

Based on systematic ablation experiments, the 8 odd kernel sizes [1,3,5,7,9,11,13,15] achieve the optimal balance among performance, computational efficiency, and interpretability, and are our recommended configuration for PEMS-08 and all other datasets.

S.3.6 Dataset-Specific Hyperparameter Rationale

We adopt differentiated hyperparameter configurations tailored to dataset characteristics. COVID datasets, with limited sample sizes (86 training samples and 30 test samples), employ smaller model capacity ($d_model = 32$) and single Belt Block layer to prevent overfitting. Learning rates are adjusted according to network scale and convergence characteristics, with PEMS04 using the smallest rate (0.0001) due to its largest network (307 nodes) to avoid gradient explosion. COVID data intervention frequency is set to 2 (every 2 batches), higher than traffic datasets’ 10, to enhance directed dependency learning in small-sample scenarios. Table 2 of the main paper appendix presents hyperparameter configurations across all five datasets.

S.3.7 Intervention Implementation Details

Intervention node selection: At the beginning of each batch, we use $\text{torch.randperm}(N)[K]$ to randomly select K nodes ($K=30$ for traffic data, $K=25$ for epidemic and DREAM-3 data). The selection process is independent of the current model state and time period.

Intervention value generation: For the selected node i , the intervention value is generated through $v_i = \tanh(g_i(c_i))$, where g_i is a node-specific learnable function and c_i is noise sampled from a standard normal distribution.

Intervention implementation: During GRU propagation, the input of the intervened node is forced to be v_i , and its incoming edges are truncated, simulating the semantics of the do-operator.

Validity calculation: The intervention effect $\Delta \hat{y}_j$ is calculated by comparing predictions before and after intervention, then compared with C_{ij} to obtain the validity score (Equation 5 of the main paper). The entire process is differentiable and trained end-to-end.

Table 10: Detailed Performance Comparison of Different Kernel Configurations

Config	Horizon 3			Horizon 6			Horizon 12			Average			Time (s/ep)
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	
3	9.79	14.62	7.30	13.24	20.88	8.41	17.99	27.81	10.25	14.33	22.73	8.71	52.7
4	8.76	12.08	7.55	13.35	20.06	9.62	17.46	26.51	11.65	14.21	21.81	10.16	57.9
8	9.44	12.98	8.82	13.13	19.66	10.41	15.18	23.37	11.42	13.29	20.24	10.55	52.7
12	—	—	—	—	—	—	—	—	—	—	—	—	~2700

S.4 LLM Fine-tuning Details

This section provides the complete data construction, training configuration, and convergence analysis underlying the LLM-enhanced retrieval module summarized in Appendix C of the main paper. Our LLM is specifically fine-tuned on spatiotemporal data using a dataset of ~100,000 time series–prediction pairs from PEMS03. Each sample contains 240 historical steps (one month of 5-minute data) with 12-step predictions. This window design matches our tri-period architecture: hourly (recent hours), daily (past 7 days), and weekly (past 4 weeks). Similarity is computed with weights $w_{cos} = 0.7$ and $w_{euc} = 0.3$ to balance semantic matching.

S.4.1 Dataset Construction and Preprocessing

The construction of the fine-tuning dataset is critical for enabling the LLM to understand time series prediction semantics. We systematically extracted approximately 100,000 training samples from raw traffic flow data, with each sample containing 240 historical observations and their corresponding 12-step prediction targets. The data is organized in ChatML format with the following structure:

```
{
  "messages": [
    {
      "role": "user",
      "content": "[240-step historical sequence],
                  Predict traffic flow in next 12 steps."
    },
    {
      "role": "assistant",
      "content": "[12-step future predictions]"
    }
  ]
}
```

This conversational format enables the model to understand the prediction task naturally. During preprocessing, all values are normalized to $[0, 1]$ and retain 4 decimal places. Given LLM input token limitations (max_length=2048), we employ backward truncation to prioritize recent observations. This strategy is justified because: (1) recent data carries more predictive information in time series forecasting; (2) early patterns appear repeatedly in training, allowing effective pattern learning.

S.4.2 Training Configuration and Optimization Strategy

The fine-tuning process employs a parameter-efficient training strategy. The specific configurations are shown in Table 11.

Table 11: Fine-tuning hyperparameters and configurations

Hyperparameter	Value	Description
Batch size	16	Effective batch per GPU
Gradient accumulation	2	Equivalent batch size of 32
Initial learning rate	2e-5	AdamW optimizer learning rate
Training epochs	5	Prevent overfitting
Warmup steps	200	Linear warmup strategy
Max sequence length	2048	Context window limit
Weight decay	0.01	L2 regularization coefficient

Training employs gradient checkpointing [1] to reduce memory usage while utilizing mixed precision [6] training to improve efficiency. The learning rate schedule adopts a linear warmup followed by constant strategy to ensure stable model convergence.

S.4.3 Training Process and Convergence Analysis

During the training process, model perplexity steadily decreased from an initial 8.3 to 2.1, indicating successful learning of the semantic representation of spatiotemporal data. The validation loss curve shows the model stabilized after epoch 3 without overfitting phenomena. This is attributed to our early stopping strategy and moderate regularization settings.

The fine-tuned model demonstrates powerful recognition capabilities for temporal patterns. Even when the absolute value ranges of input sequences differ, the model can accurately identify typical traffic patterns such as “morning peak–off-peak–evening peak” and retrieve the most similar historical patterns for matching. For instance, the model successfully recognizes that a sequence with pattern “early peak – stable flow – evening peak” semantically matches similar daily patterns despite occurring on different calendar days. This semantic understanding capability forms the core foundation of our intelligent retrieval mechanism.

S.5 Evaluation Metrics Definition

We employ three standard evaluation metrics to comprehensively assess prediction performance:

$$\text{MAE} = \frac{1}{N \times H} \sum_{i=1}^N \sum_{t=1}^H |y_i^{(t)} - \hat{y}_i^{(t)}| \quad (7)$$

$$\text{RMSE} = \sqrt{\frac{1}{N \times H} \sum_{i=1}^N \sum_{t=1}^H (y_i^{(t)} - \hat{y}_i^{(t)})^2} \quad (8)$$

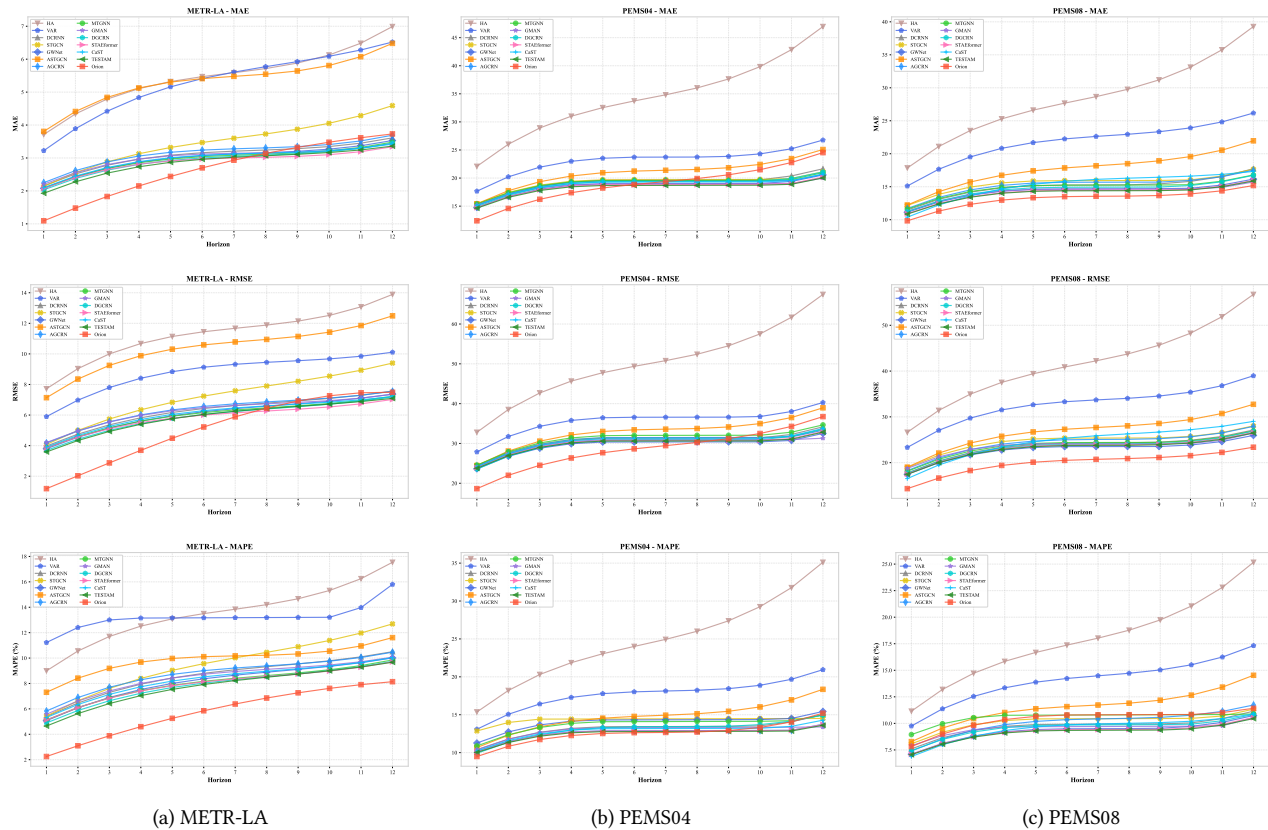


Figure 1: Performance comparison across 12 prediction horizons on three traffic datasets.

$$\text{MAPE} = \frac{100\%}{N \times H} \sum_{i=1}^N \sum_{t=1}^H \frac{|y_i^{(t)} - \hat{y}_i^{(t)}|}{\max(|y_i^{(t)}|, \epsilon_{\text{thresh}})} \quad (9)$$

where $y_i^{(t)}$ and $\hat{y}_i^{(t)}$ represent the ground truth and prediction for node i at time t respectively, N is the number of nodes, H is the prediction horizon, and $\epsilon_{\text{thresh}} = 5$ is a threshold to handle near-zero values in traffic datasets.

MAE measures average absolute deviation, providing a linear penalty for errors and being robust to outliers. RMSE applies quadratic penalty to errors, making it more sensitive to large deviations and suitable for applications where large errors are particularly undesirable. MAPE provides a scale-independent percentage error measure, enabling fair comparison across different value ranges, though it requires careful handling of near-zero values.

S.6 Data Preprocessing

Traffic datasets undergo preprocessing including IQR-based outlier detection [9] and Savitzky-Golay smoothing [7] to handle sensor noise, while epidemic datasets retain raw values due to their daily aggregation nature. The preprocessing pipeline consists of: (1) missing value imputation using linear interpolation for gaps less than 3 time steps and forward-filling for longer gaps; (2) outlier detection using the interquartile range method with a factor of 1.5; (3) noise reduction through Savitzky-Golay filtering with window length

7 and polynomial order 2 for traffic data only; (4) normalization using training set statistics for all splits to prevent data leakage.

S.7 Supplementary Experimental Results

S.7.1 Performance Comparison Visualization

Figure 1 visualizes complete 12-horizon performance trajectories of all methods across three traffic datasets, revealing distinct clustering. On METR-LA, statistical methods (HA, VAR) form the highest error cluster with steep degradation curves. Deep learning methods converge to a competitive band between MAE 3.0-4.0, with marginal differences reflecting architectural variations. Orion establishes a clearly lower trajectory, maintaining advantages across all horizons. The visualization highlights Orion’s superior early-horizon performance, where directed dependencies exhibit strongest predictive power before uncertainty affects long-term predictions.

S.7.2 METR-LA Supplementary Analysis

Extended Prediction Results Analysis. Figure 2 demonstrates Orion’s 600-step prediction results on 4 representative nodes from METR-LA. Node 0 displays distinct V-shaped valleys (flow dropping from 60 to 20), which Orion almost perfectly reproduces. Node 1’s smoother pattern achieves the best MAE of 2.323. Node 2 exhibits sudden changes—flow plummeting from 80 to 20 or surging from

40 to 100—with Orion accurately capturing both timing and magnitude. This sensitivity stems from multi-scale temporal convolutions detecting both short-term disruptions and long-term trends. Node 3 presents oscillations between 40-70 combining regular periodicity with irregular perturbations; Orion maintains 6.4% MAPE even in complex segments. At extreme valleys approaching flow of 10, the model shows slight overestimation but quickly self-corrects, demonstrating TE-CausGAT's generalization capability.

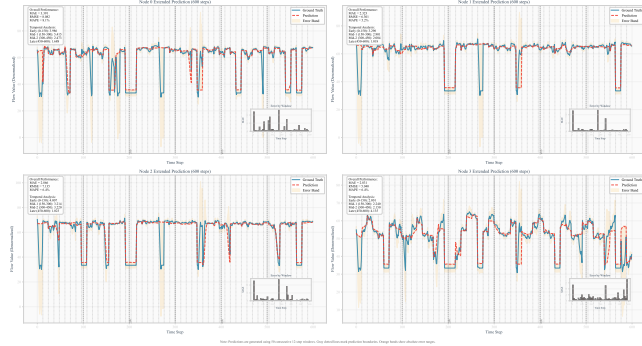


Figure 2: Extended prediction analysis for 4 representative nodes over 600 time steps through 50 consecutive 12-step sliding windows. Blue lines show ground truth, red dashed lines show predictions, orange bands indicate error ranges, and gray dotted lines mark prediction window boundaries.

Temporal Attention Weight Analysis. Temporal attention weight analysis (Figure 3) reveals the model's sophisticated temporal modeling mechanism. The global average attention heatmap displays clear diagonal dominance with maximum weight reaching 0.1179, concentrated on adjacent time steps, embodying traffic flow's Markovian properties. More interestingly, secondary attention peaks appear at t-6 and t-8 positions, corresponding exactly to historical data from 1-1.5 hours ago (12 steps $\times$ 5 minutes). This "bi-modal" attention pattern enables the model to simultaneously capture immediate dynamics and hourly periodicity, perfectly matching urban traffic temporal characteristics.

Multi-Period Fusion Weight Analysis. Multi-period fusion weight analysis (Figure 4) reveals surprising yet reasonable patterns. Weekly period data receives the highest weight of 48.7%, far exceeding hourly (26.4%) and daily (25.0%) periods. This finding highly aligns with METR-LA's actual traffic characteristics—Los Angeles traffic shows significant differences between weekdays and weekends, with weekly patterns' importance surpassing daily variations. This adaptive weight allocation mechanism enables Orion to automatically adjust multi-scale information fusion strategies based on different datasets' characteristics.

S.7.3 PEMS Datasets Supplementary Analysis

Since the PEMS04 and PEMS08 datasets lack precise geographic coordinates for sensors, we cannot perform spatial causality visualization analysis similar to METR-LA. This section focuses on detailed visualizations of prediction performance and attention mechanisms on these two datasets.

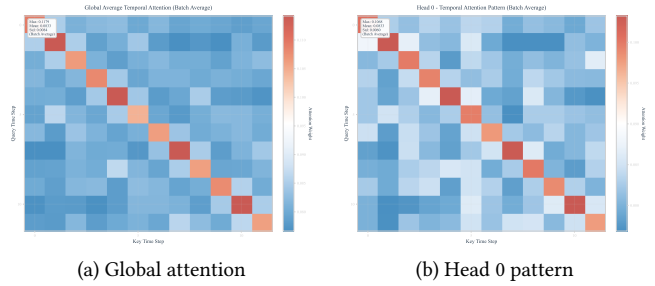


Figure 3: Temporal attention patterns in METR-LA. Maximum weight reaches 0.1179 with standard deviation of only 0.0084, demonstrating high stability.

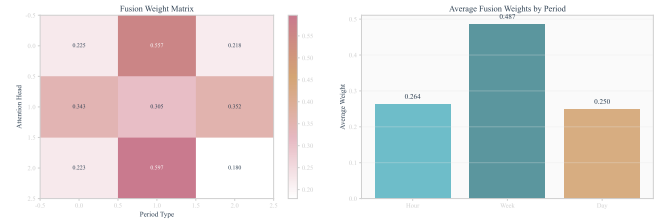


Figure 4: Multi-period fusion weight analysis.

S.7.3.1 Extended Prediction Performance. Figures 5 and 6 present prediction results on the PEMS datasets, which exhibit markedly different traffic characteristics and pose distinct prediction challenges.

PEMS08 Regular Pattern Recognition. PEMS08 displays highly regular traffic patterns shaped by stable commuting cycles. Nodes 0 and 1 exhibit clear rectangular wave patterns—rapidly switching between high-flow plateaus and low-flow valleys with sharp transition boundaries. Orion precisely identifies these binary state transitions, with the red prediction line accurately responding at each transition point without delay or overshoot. Particularly for Node 0's multiple deep valleys (flow dropping to near zero), the model not only predicts the extreme low values at valley bottoms but also accurately captures their duration and recovery slope. Node 2 demonstrates more gradual, sinusoidal-like fluctuations, where Orion similarly achieves good fitting with only slight underestimation at individual peaks.

PEMS04 Sparse Flow Challenges. The most distinctive feature of the PEMS04 dataset is extreme flow sparsity—multiple nodes maintain near-zero flow for prolonged periods, likely reflecting low-traffic ramps or auxiliary lanes. Nodes 0 and 2 frequently exhibit complete flow cessation, posing severe challenges for prediction models that rely on continuous signals. Orion demonstrates remarkable adaptability: accurately maintaining zero predictions during zero-flow periods and responding promptly when flow resumes. However, Node 3's anomalous MAPE value exposes inherent limitations of the evaluation metric—when ground truth values approach zero, even minimal absolute errors can cause enormous relative errors. Nevertheless, visually, the red prediction line still closely adheres to the blue ground truth curve, indicating the model's actual performance far exceeds what the MAPE metric suggests.

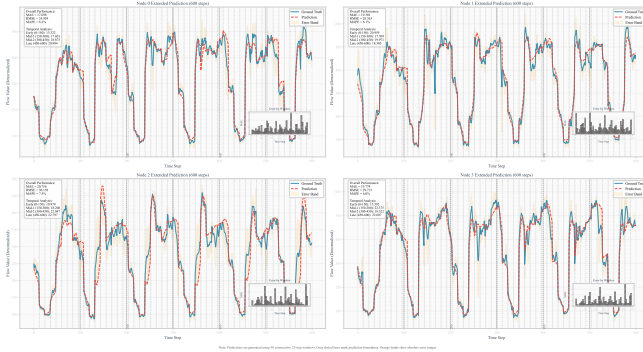


Figure 5: Extended prediction results on PEMS08 for nodes 0-3. Predictions generated using 50 consecutive 12-step sliding windows for 600 time steps. Gray dotted lines mark prediction boundaries and orange bands show absolute error ranges.

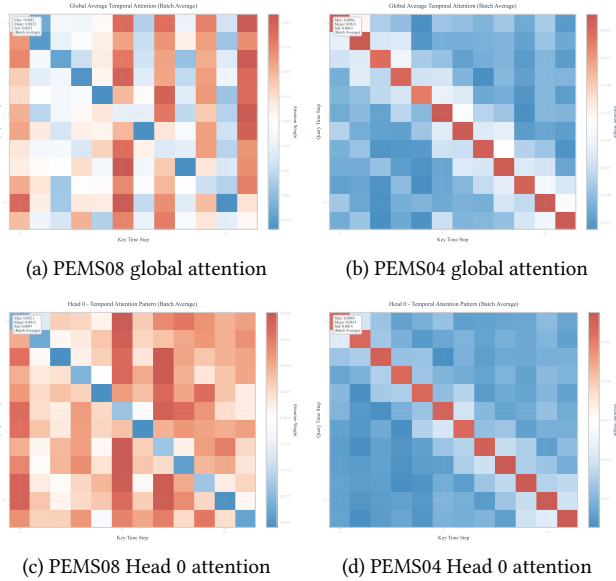


Figure 7: Temporal attention patterns on PEMS datasets.

S.7.3.2 Temporal Attention Patterns. Figure 7 illustrates temporal attention weight distributions for both datasets. PEMS08’s global average attention presents a clear diagonal-dominant pattern with maximum weight 0.0882 on adjacent time steps and standard deviation 0.0023, reflecting high stability. Head 0’s pattern reveals richer structure with a secondary peak at $t-6$, corresponding to data from 1 hour ago (12 steps $\times$ 5 minutes), capturing hourly periodicity. PEMS04’s attention is more concentrated with maximum weight 0.0864 and standard deviation 0.0010. This tighter distribution may relate to PEMS04’s shorter time span (2 months), with the model relying more on recent observations. The reinforced diagonal pattern indicates stronger Markovian properties. Notably, fusion weights for both datasets remain consistent across attention heads, with minimal differences between heads, validating the stability and robustness of our multi-head fusion mechanism.

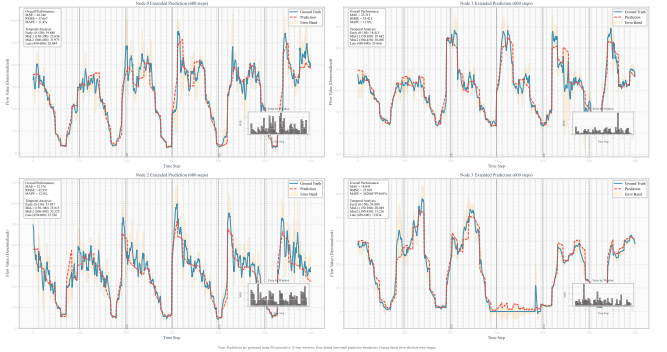


Figure 6: Extended prediction results on PEMS04 dataset for nodes 0-3, showing 600-step forecasts via 50 sliding windows. Note the anomalous MAPE for Node 3 due to near-zero ground truth values.



Figure 8: Multi-period fusion weight distribution. Top: PEMS08. Bottom: PEMS04.

S.7.3.3 Multi-Period Fusion Weights. The multi-period fusion weight distribution provides insights into temporal scale preferences across datasets with distinct periodic structures. As shown in Figure 8, on PEMS08, average fusion weights for hour, week, and day periods are 0.337, 0.322, and 0.341 respectively, showing relatively balanced distribution with mild preference for daily and hourly patterns. The batch-averaged standard deviation is 0.015, indicating certain dynamics in weight allocation across time segments and adaptive responsiveness to local pattern shifts. PEMS04 exhibits more uniform weight allocation, with all three periods extremely close to 0.333 and standard deviation of only 0.001. This near-equal partition suggests the model considers information from all three temporal scales equally important, possibly reflecting stronger regularity and more homogeneous periodicity in PEMS04’s traffic patterns, where periodic features at hourly, daily, and weekly scales all provide comparably valuable predictive information.

S.8 Extended Reproducibility, Limitations, and Ethics

This section reproduces the full original Reproducibility Statement, Limitations, and Ethics Statement of the paper for completeness.

S.8.1 Reproducibility Statement

To facilitate independent verification, we publicly release all experiment code, configuration files, and data preprocessing scripts at the anonymous repository linked in Section 1. Our implementation is built on PyTorch 2.0+ with experiments conducted on NVIDIA A100-40GB GPUs (traffic datasets), RTX 2080Ti-22GB GPUs (epidemic datasets), and RTX 5880 Ada-48GB GPU (LLM retrieval). Deterministic execution is ensured by fixing all random seeds to 42 across Python, NumPy, PyTorch, and CUDA backends. Every hyperparameter is exhaustively documented in the appendix, including three-stage training schedules (Table 2 of the main paper appendix and Table 7 of this supplementary appendix), intervention details (Appendix B of the main paper), and preprocessing procedures (Section S.6 of this supplementary appendix). All five datasets are publicly available benchmarks with standard community splits. The repository includes a comprehensive README.md with step-by-step instructions for environment setup, data preparation, and end-to-end training.

S.8.2 Limitations

Although Orion achieves strong results across five benchmarks, several practical constraints should be noted. The intervention-based validation requires extra forward passes during training, adding roughly 15–20% wall-clock overhead compared to a correlation-only baseline; in practice this is kept manageable through the intervention frequency hyperparameter and by restricting the causal learning phase to 10 of the 100 total epochs, with no additional cost at inference time. On PEMS04, which covers only 2 months with 307 nodes, the multi-period architecture tends to overfit at longer horizons—a data scarcity issue rather than a modeling flaw, as the same design yields consistent gains on METR-LA (4 months) and PEMS08 (170 nodes). Our polynomial DAG constraint penalizes only 2- and 3-cycles and thus cannot theoretically rule out longer cycles, though empirical verification via topological sorting detects none on PEMS-08 at threshold 0.1 (Table 3). The TinyLlama retrieval module has $O(N \times H)$ complexity, and due to limited GPU memory availability, full-network LLM-based retrieval remains computationally prohibitive for datasets with hundreds of nodes, restricting current evaluation to 5-node subsets obtained via K-means; all primary results reported in Table 2 of the main paper rely on fixed-period retrieval and are therefore unaffected by this constraint.

S.8.3 Ethics Statement

This study uses exclusively publicly available datasets containing no personally identifiable information. The traffic datasets (PEMS04, PEMS08, METR-LA) consist of aggregated loop-detector readings recording only vehicle counts, occupancy ratios, and speeds; the COVID-19 datasets (CA-COVID, TX-COVID) provide county-level hospitalization statistics from Dong et al. [2]; and the DREAM-3 dataset [5] comprises entirely synthetic gene expression trajectories.

All usage complies with respective distribution licenses and privacy regulations. Orion is designed as a decision-support tool whose forecasts and dependency structures are intended to inform—rather than replace—human judgment in traffic management and public health planning. No human subjects were involved at any stage of this research, and no demographic or sensitive attributes are modeled or inferred.

References

- [1] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [2] Ensheng Dong, Hongru Du, and Lauren Gardner. 2020. An interactive web-based dashboard to track COVID-19 in real time. *The Lancet infectious diseases* 20, 5 (2020), 533–534.
- [3] Clive WJ Granger. 1969. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica: journal of the Econometric Society* (1969), 424–438.
- [4] Saurabh Khanna and Vincent YF Tan. 2019. Economy statistical recurrent units for inferring nonlinear granger causality. *arXiv preprint arXiv:1911.09879* (2019).
- [5] Aviv Madar, Alex Greenfield, Eric Vanden-Eijnden, and Richard Bonneau. 2010. DREAM3: network inference using dynamic context likelihood of relatedness and the inferelator. *PLoS one* 5, 3 (2010), e9803.
- [6] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).
- [7] Jean Steinier, Yves Termonia, and Jules Deltour. 1972. Smoothing and differentiation of data by simplified least square procedure. *Analytical chemistry* 44, 11 (1972), 1906–1909.
- [8] Alex Tank, Ian Covert, Nicholas Foti, Ali Shojaie, and Emily B Fox. 2021. Neural granger causality. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2021), 4267–4279.
- [9] John Wilder Tukey et al. 1977. *Exploratory data analysis*. Vol. 2. Springer.